Building a Movie Recommendation Engine

with Naïve Bayes

Machine Learning Practice With Python

Siksha 'O' Anushandhan, ITER

Center for Data Science

Table of Contents

Classification in Machine Learning

- **Classification in ML**: A type of supervised learning where the model learns to map **features (input data)** to **labels/classes (categories)** based on a training dataset.
- **Goal**: Learn a **general rule** from given observations and their associated categorical outputs to classify future data correctly.
- 🎓 **Training Phase**: The model uses **features** + **labels** from known data to build a **trained classification model**.
- 🪄 **Prediction Phase**: When **new, unseen data** arrives, the trained model predicts the **class membership** based on input features.

Classification in Machine Learning

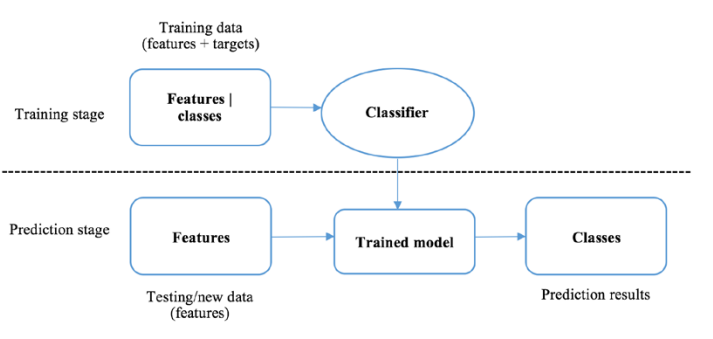


Figure: **The training and prediction stages in classification**

Types of Classification (Binary Classification)

Binary Classification: Categorizes data into **two possible classes** e.g., spam vs. not spam in email filtering, churn vs. retain in customer prediction systems.

Real-World Applications

Marketing/Advertising
Predict if an online ad will be **clicked** using user data (cookies, browsing history).

Customer Analytics
Identify potential **churners** using CRM and behavioral data.

Biomedical Use
Assist early diagnosis e.g., classify patients into **high-risk vs. low-risk** groups for diseases like cancer from MRI images.

Types of Classification (Multiclass Classification)

Multiclass Classification: Assigns observations to **more than two classes**, unlike binary classification with only two.

Classic Example

Handwritten digit recognition (digits **0-9**) historically important and used in automatic **ZIP code reading**.

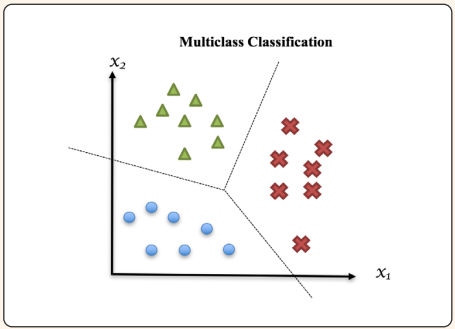
MNIST Dataset

Classic **benchmark** for testing multi-class classifiers on handwritten digits.

60,000 training images

10,000 test images

Visual Example



Exploring Naïve Bayes

- **Naïve Bayes** is a **probabilistic classifier** that computes the probability of a data sample belonging to each class based on its **predictive features** (attributes or signals).
- It produces a **probability distribution** over all classes and selects the **most likely class** for prediction.
- From the resulting probability distribution, we can conclude the **most likely class** that the data sample is associated with.

What Naïve Bayes does specifically, as its name indicates:

Bayes: Uses Bayes' theorem to relate the probability of observed features given a class to the probability of the class given those features.

⋆ Naïve: Assumes all predictive features are mutually independent to simplify probability calculations.

Bayes' Theorem with Examples

Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

where, $P(B \mid A)$ is the probability of B given A, while $P(A)$ and $P(B)$ are the probabilities that A and B occur, respectively.

Exercise:

Given two coins, one is unfair, with 90% of flips getting a head and 10% getting a tail, while the other one is fair. Randomly pick one coin and flip it. What is the probability that this coin is the unfair one, if we get a head?

Solution

Event Definitions:

- A = event of picking the **unfair** coin
- B = The flip shows head
- To find: $P(A|B)$
- Given: $P(A) = 0.5$ $P(\neg A) = 0.5$ $P(B \mid A) = 0.9$ $P(B \mid \neg A) = 0.5$

$$P(B) = P(B \mid A)P(A) + P(B \mid \neg A)P(\neg A) = 0.70$$
$$\Rightarrow P(A \mid B) \approx 0.64$$

Bayes' Theorem with Examples

Exercise:

Suppose a physician reported the following cancer screening test scenario among 10,000 people:

	Cancer	No Cancer	Total
Test Positive	80	900	980
Test Negative	20	9000	9020
Total	100	9900	10000

If a person test positive, what is the probability that actually have cancer?

Bayes' Theorem with Examples

Exercise:

Suppose a physician reported the following cancer screening test scenario among 10,000 people:

	Cancer	No Cancer	Total
Test Positive	80	900	980
Test Negative	20	9000	9020
Total	100	9900	10000

If a person test positive, what is the probability that actually have cancer?

$$P(Cancer \mid test + ve) = \frac{P(test + ve \mid Cance) \times P(Cancer)}{P(test + ve)} = \frac{\frac{80}{100} \times \frac{100}{10000}}{\frac{980}{10000}}$$

$$= 0.08163$$

Exercise

Exercise:

1. Three machines A, B, and C in a factory account for 35%, 20%, and 45% of bulb production. The fraction of defective bulbs produced by each machine is 1.5%, 1%, and 2%, respectively. A bulb produced by this factory was identified as defective, which is denoted as event D. What are the probabilities that this bulb was manufactured by machines A, B, and C, respectively.
2. At an airport, 1% of passengers carry prohibited items. A security scanner correctly identifies a passenger with a prohibited item 95% of the time, but it also falsely alarms 5% of passengers who dont carry prohibited items. If a passenger triggers the alarm, what is the probability that they actually have a prohibited item?

Naïve Bayes Classifier mechanism overview

Given a data sample $\boldsymbol{x}$ with n features $x_1, x_2, \ldots, x_n$ (where $\boldsymbol{x}$ represents a feature vector and $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$).

The **goal** of Nave Bayes is to determine the probabilities that this sample belongs to each of K possible classes $y_1, y_2, \ldots, y_K$, which is $P(y_k \mid \boldsymbol{x})$ for $k = 1, 2, \ldots, K$.

Bayes Theorem:

$$P(y_k \mid \boldsymbol{x}) = \frac{P(\boldsymbol{x} \mid y_k) \times P(y_k)}{P(\boldsymbol{x})}$$

- $P(y_k)$: **Prior probability** of class k
- $P(\boldsymbol{x} \mid y_k)$, or equivalently $P(x_1, x_2, \ldots, x_n \mid y_k)$, is the **joint distribution** of n features given that the sample belongs to class y_k.
- $P(y_k \mid \boldsymbol{x})$: **Posterior probability** of class given features
- $P(\boldsymbol{x})$: **Evidence** (normalization factor)

Naïve Independence Assumption

Assumption: Features are **conditionally independent**

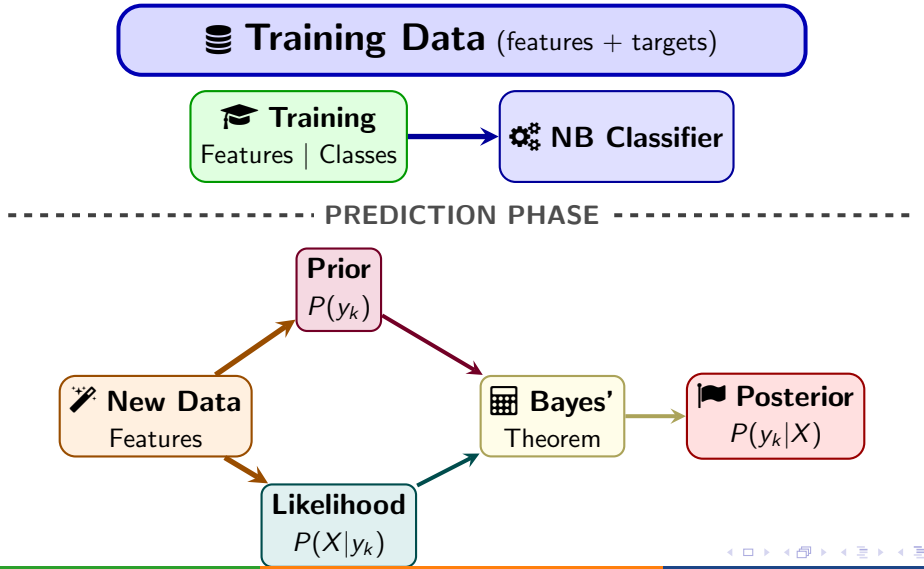
The joint conditional distribution of **n** features can be expressed as:

$$P(\mathbf{x} \mid y_k) = \prod_{i=1}^{n} P(x_i \mid y_k)$$

Hence, the posterior probability is proportional to:

$$P(y_k \mid \mathbf{x}) \propto P(y_k) \prod_{i=1}^{n} P(x_i \mid y_k)$$

Naïve Bayes: Training & Prediction Workflow



Example

Example

Given four (pseudo) users, whether they like each of three movies, m_1, m_2, m_3 (indicated as 1 or 0), and whether they like a target movie (denoted as event Y) or not (denoted as event N), as shown in the following table, we are asked to predict how likely it is that another user will like that movie:

ID	m_1	m_2	m_3	Whether the user likes the target movie
1	0	1	1	Y
2	0	0	1	N
3	0	0	0	Y
4	1	1	0	Y
Testing case				
5	1	0	1	?

Naïve Bayes Classification Example

Training Data

$P(Y) = \frac{3}{4}$, $P(N) = \frac{1}{4}$
Y = {users 1,3,4}, N = {user 2}

Naïve Bayes Classification Example

Training Data

$P(Y) = \frac{3}{4}$, $P(N) = \frac{1}{4}$
Y = {users 1,3,4}, N = {user 2}

Class Y (3 examples)

$P(m_1 = 1|Y) = \frac{1}{3}$ (user 4)
$P(m_2 = 0|Y) = \frac{1}{3}$ (user 3)
$P(m_3 = 1|Y) = \frac{1}{3}$ (user 1)
$P(\mathbf{x}|Y) = \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{27}$

$$P(Y) \cdot P(\mathbf{x}|Y) = \frac{3}{4} \cdot \frac{1}{27} = \boxed{\frac{1}{36}}$$

Naïve Bayes Classification Example

Training Data

$P(Y) = \frac{3}{4}$, $P(N) = \frac{1}{4}$
Y = {users 1,3,4}, N = {user 2}

Class Y (3 examples)

$P(m_1 = 1|Y) = \frac{1}{3}$ (user 4)
$P(m_2 = 0|Y) = \frac{1}{3}$ (user 3)
$P(m_3 = 1|Y) = \frac{1}{3}$ (user 1)
$P(\mathbf{x}|Y) = \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{27}$

$$P(Y) \cdot P(\mathbf{x}|Y) = \frac{3}{4} \cdot \frac{1}{27} = \boxed{\frac{1}{36}}$$

Class N (1 example)

Problem: $P(m_1 = 1|N) = 0$
(user 2 has $m_1 = 0$)
$P(\mathbf{x}|N) = 0 \cdot P(m_2 = 0|N) \cdot P(m_3 = 1|N)$
$P(\mathbf{x}|N) = \boxed{0}$

Naïve Bayes Classification Example

Training Data

$P(Y) = \frac{3}{4}$, $P(N) = \frac{1}{4}$
Y = {users 1,3,4}, N = {user 2}

Class Y (3 examples)

$P(m_1 = 1|Y) = \frac{1}{3}$ (user 4)
$P(m_2 = 0|Y) = \frac{1}{3}$ (user 3)
$P(m_3 = 1|Y) = \frac{1}{3}$ (user 1)
$P(\mathbf{x}|Y) = \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{3} = \frac{1}{27}$

$$P(Y) \cdot P(\mathbf{x}|Y) = \frac{3}{4} \cdot \frac{1}{27} = \boxed{\frac{1}{36}}$$

Class N (1 example)

Problem: $P(m_1 = 1|N) = 0$
(user 2 has $m_1 = 0$)
$P(\mathbf{x}|N) = 0 \cdot P(m_2 = 0|N) \cdot P(m_3 = 1|N)$
$P(\mathbf{x}|N) = \boxed{0}$

Final Result

After normalization:
$\boxed{P(Y|\mathbf{x}) = 1}$
$\boxed{P(N|\mathbf{x}) = 0}$
Classification: Y

Implementing Naïve Bayes with scikit-learn

- Define dataset

```
1 import numpy as np
2 X_train = np.array([[0, 1, 1],  [0, 0, 1], [0, 0, 0], [1, 1, 0
          ]])
3 Y_train = ['Y', 'N', 'Y', 'Y']
4 X_test = np.array([[1, 0, 1]])
```

- Create Naïve-Bayes model from scikit-learn

Implementing Naïve Bayes with scikit-learn

- Define dataset

```
1 import numpy as np
2 X_train = np.array([[0, 1, 1],  [0, 0, 1], [0, 0, 0], [1, 1, 0
          ]])
3 Y_train = ['Y', 'N', 'Y', 'Y']
4 X_test = np.array([[1, 0, 1]])
```

- Create Naïve-Bayes model from scikit-learn

```
1 from sklearn.naive_bayes import BernoulliNB
2 clf = BernoulliNB(alpha=1.0, fit_prior=True)
```

- Train and predict class

Implementing Naïve Bayes with scikit-learn

- Define dataset

```
import numpy as np
X_train = np.array([[0, 1, 1],  [0, 0, 1], [0, 0, 0], [1, 1, 0
        ]])
Y_train = ['Y', 'N', 'Y', 'Y']
X_test = np.array([[1, 0, 1]])
```

- Create Naïve-Bayes model from scikit-learn

```
from sklearn.naive_bayes import BernoulliNB
clf = BernoulliNB(alpha=1.0, fit_prior=True)
```

- Train and predict class

```
clf.fit(X_train, Y_train)
pred = clf.predict(X_test)
print('[scikit-learn] Prediction:', pred)
```

```
[scikit-learn] Prediction: ['Y']
```

Building a Movie Recommender with Naïve Bayes

MovieLens Small Dataset

Download: `ml-latest-small.zip` (1 MB)
files.grouplens.org/datasets/movielens/

Dataset Statistics:

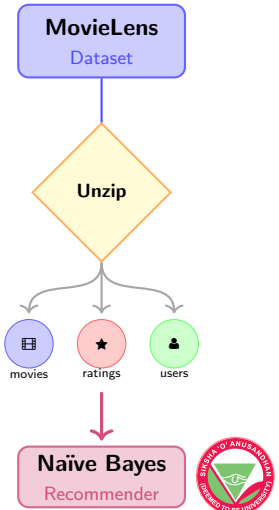
100,836 ratings **6,10** users **9724** movies

Dataset Files

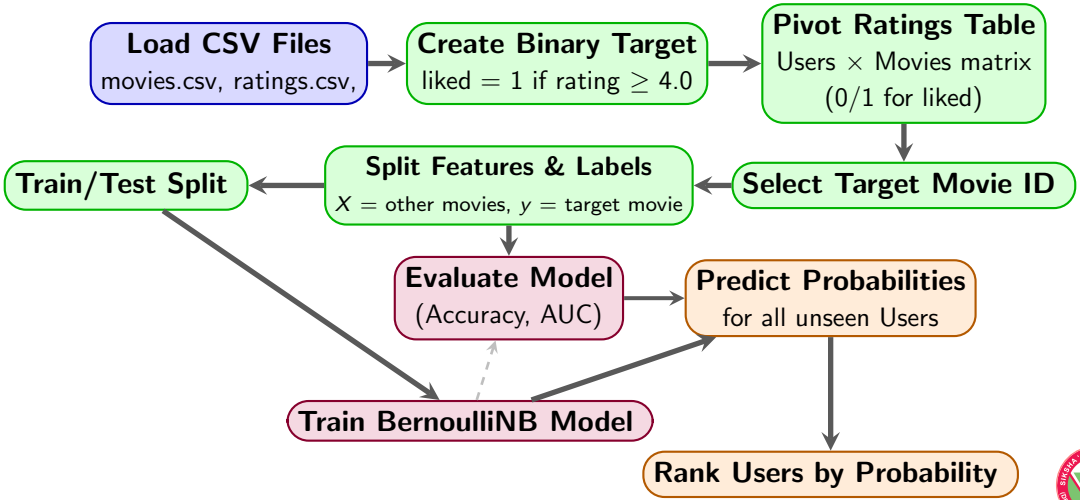
`movies.csv`	movieId::title::genres
`ratings.csv`	userId::movieId::rating::timestamp *(primary file for this model)*
`README.txt`	Documentation

Target

Predict a particular movie will recommend to which user.



Movie Recommendation System Workflow



Building a Movie Recommender with Naïve Bayes

- Import data (ratings.csv, movies.csv)

```
1 import numpy as np
2 import pandas as pd
3 movies = pd.read_csv("movies.csv")
4 ratings = pd.read_csv("ratings.csv")
5 print(movies.head())
6 print(ratings.head())
```

- Merge `movies` and `ratings` on `movieId` [Optional]

Building a Movie Recommender with Naïve Bayes

- Import data (ratings.csv, movies.csv)

```
import numpy as np
import pandas as pd
movies = pd.read_csv("movies.csv")
ratings = pd.read_csv("ratings.csv")
print(movies.head())
print(ratings.head())
```

- Merge `movies` and `ratings` on `movieId` [Optional]

```
df = pd.merge(ratings, movies, on='movieId')
df.head()
```

Building a Movie Recommender with Naïve Bayes

- Now, lets see how many unique users and movies are in the dataset

Building a Movie Recommender with Naïve Bayes

- Now, lets see how many unique users and movies are in the dataset

```
n_users = df['userId'].nunique()
n_movies = df['movieId'].nunique()
print(f"Number of users: {n_users}")
print(f"Number of movies: {n_movies}")
```

```
Number of users: 610
Number of movies: 9724
```

- Check which rating is rated by how many users

Building a Movie Recommender with Naïve Bayes

- Now, lets see how many unique users and movies are in the dataset

```
n_users = df['userId'].nunique()
n_movies = df['movieId'].nunique()
print(f"Number of users: {n_users}")
print(f"Number of movies: {n_movies}")
```

```
Number of users: 610
Number of movies: 9724
```

- Check which rating is rated by how many users

```
values, counts = np.unique(df['rating'], return_counts=True)
for value, count in zip(values, counts):
print(f'Number of rating {value}: {count}')
```

```
Number of rating 0.5: 1370
Number of rating 1.0: 2811
Number of rating 1.5: 1791
Number of rating 2.0: 7551
Number of rating 3.0: 20047
Number of rating 3.5: 13136
Number of rating 4.0: 26818
Number of rating 4.5: 8551
```

Building a Movie Recommender with Naïve Bayes

- Create a new column "`liked`" with values 1 if **rating** is ≥ 4.

Building a Movie Recommender with Naïve Bayes

- Create a new column "liked" with values 1 if **rating** is ≥ 4.

```
df['liked'] = (df['rating'] >= 4.0).astype(int)
df
```

- Create a dataframe with columns as movie ID and index as User ID and values are whether the user liked that movie or not (0/1) [you can use `pivot_table`]

Building a Movie Recommender with Naïve Bayes

- Create a new column "liked" with values 1 if **rating** is ≥ 4.

```
df['liked'] = (df['rating'] >= 4.0).astype(int)
df
```

- Create a dataframe with columns as movie ID and index as User ID and values are whether the user liked that movie or not (0/1) [you can use pivot_table]

```
userId_movieId_rating = df.pivot_table(index='userId',
        columns='movieId', values='liked')
userId_movieId_rating.fillna(0, inplace=True)
userId_movieId_rating
```

- Fix a movie ID, and consider that movie ID column as target and remaining all as features.

Building a Movie Recommender with Naïve Bayes

- Create a new column "liked" with values 1 if **rating** is ≥ 4.

```
df['liked'] = (df['rating'] >= 4.0).astype(int)
df
```

- Create a dataframe with columns as movie ID and index as User ID and values are whether the user liked that movie or not (0/1) [you can use pivot_table]

```
userId_movieId_rating = df.pivot_table(index='userId',
        columns='movieId', values='liked')
userId_movieId_rating.fillna(0, inplace=True)
userId_movieId_rating
```

- Fix a movie ID, and consider that movie ID column as target and remaining all as features.

```
target_movie = 2858
Y = userId_movieId_rating[target_movie]
X = userId_movieId_rating.drop(columns=[target_movie])
```

Building a Movie Recommender with Naïve Bayes

- Split the data into train and test set, build the `BernoulliNB` model, train the model, and generate the classification report.

Building a Movie Recommender with Naïve Bayes

- Split the data into train and test set, build the BernoulliNB model, train the model, and generate the classification report.

```
1  from sklearn.model_selection import train_test_split
2  from sklearn.naive_bayes import BernoulliNB
3  from sklearn.metrics import accuracy_score,
           classification_report, confusion_matrix
4  X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
           test_size=0.2, random_state=42)
5  clf = BernoulliNB(alpha=1.0, fit_prior=True)
6  clf.fit(X_train, Y_train)
7  pred = clf.predict(X_test)
8  print('Confusion Matrix:\n', confusion_matrix(Y_test, pred))
9  print('Accuracy:', accuracy_score(Y_test, pred))
10 print('Classification Report:\n',
           classification_report(Y_test, pred))
```

Building a Movie Recommender with Naïve Bayes

```
Confusion Matrix:
[[74  8]
[16 24]]
Accuracy: 0.8032786885245902
Classification Report:
              precision    recall  f1-score   support

         0.0       0.82      0.90      0.86        82
         1.0       0.75      0.60      0.67        40

accuracy                             0.80       122
macro avg         0.79      0.75      0.76       122
weighted avg      0.80      0.80      0.80       122
```

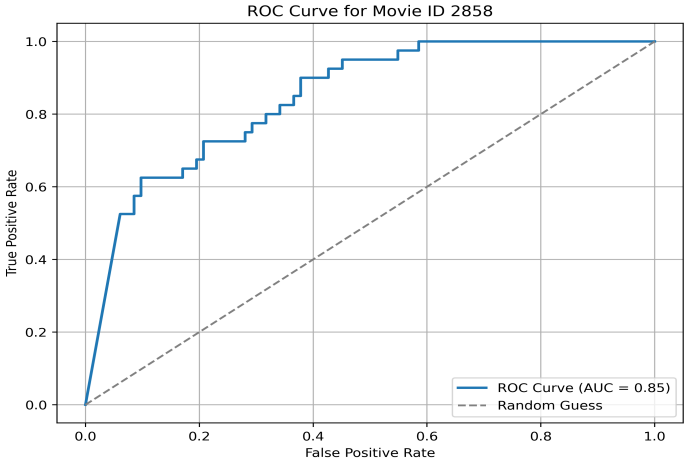
Building a Movie Recommender with Naïve Bayes

- Compute True positive rate, False positive rate, AUC, and plot AUC curve

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
y_prob = clf.predict_proba(X_test)[:, 1]  # <-- This gives
        probabilities for class 1
# Compute ROC metrics
fpr, tpr, thresholds = roc_curve(Y_test, y_prob)
auc_score = roc_auc_score(Y_test, y_prob)
```

```
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {auc_score:.2f})",
        linewidth=2)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray',
        label="Random Guess")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curve for Movie ID {target_movie}")
```

Building a Movie Recommender with Naïve Bayes



Building a Movie Recommender with Naïve Bayes

- For a particular user ID, predict class and find probability for each class.

Building a Movie Recommender with Naïve Bayes

- For a particular user ID, predict class and find probability for each class.

```
user_id = 1
x_user = userId_movieId_rating.drop(columns=[target_movie]).
        loc[user_id].values.reshape(1, -1)
predicted_preference = clf.predict(x_user)
predicted_probability = clf.predict_proba(x_user)
predicted_preference, predicted_probability
```

```
(array([1.]), array([[9.11272289e-55, 1.00000000e+00]]))
```

Building a Movie Recommender with Naïve Bayes

- Define a function which return top k users who have not watch this movie and possible like it.

Building a Movie Recommender with Naïve Bayes

- Define a function which return top k users who have not watch this movie and possible like it.

```
1 def recommend_movie(target_movie, top_k=10):
2       not_watched_user_ids = userId_movieId_rating.index[
          userId_movieId_rating[target_movie] == 0]
3       not_watched_users = X.loc[not_watched_user_ids]
4       not_watched_users_prob =
          clf.predict_proba(not_watched_users)[:, 1]
5       top_k_user_indices =
          np.argsort(not_watched_users_prob)[-top_k:][::-1]
6       top_k_user_ids = not_watched_user_ids[top_k_user_indices]
7       return top_k_user_ids,
          not_watched_users_prob[top_k_user_indices]
```

Building a Movie Recommender with Naïve Bayes

- Print movie name and top k users with probability

Building a Movie Recommender with Naïve Bayes

- Print movie name and top k users with probability

```
movie_title = movies.loc[movies.movieId == target_movie,
        "title"].values[0]
print(f"\nTop users likely to like '{movie_title}':")
top_users, top_probs =
        recommend_movie(target_movie=target_movie, top_k=10)
for user, prob in zip(top_users, top_probs):
      print(f"User ID: {user}, Probability of liking:
        {prob:.4f}")
```

```
Top users likely to like 'American Beauty (1999)':
User ID: 610, Probability of liking: 1.0000
User ID: 42, Probability of liking: 1.0000
User ID: 51, Probability of liking: 1.0000
User ID: 561, Probability of liking: 1.0000
User ID: 534, Probability of liking: 1.0000
User ID: 64, Probability of liking: 1.0000
```

Cross-validation training and find best parameter

Initiate Stratified k-fold cross validation

Cross-validation training and find best parameter

Initiate Stratified k-fold cross validation

```
1 from sklearn.model_selection import StratifiedKFold
2 alphas = np.logspace(-2, 1, 10)  # 0.01 to 10
3 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Calculate and print mean AUC for different `alpha`

Cross-validation training and find best parameter

Initiate Stratified k-fold cross validation

```
from sklearn.model_selection import StratifiedKFold
alphas = np.logspace(-2, 1, 10)  # 0.01 to 10
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

Calculate and print mean AUC for different alpha

```
alpha_scores = []
for alpha in alphas:
      clf = BernoulliNB(alpha=alpha, fit_prior= True)
      fold_scores = []
      for train_idx, val_idx in cv.split(X, Y):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = Y.iloc[train_idx], Y.iloc[val_idx]
            clf.fit(X_train, y_train)
            y_prob = clf.predict_proba(X_val)[:, 1]
            score = roc_auc_score(y_val, y_prob)
```

Cross-validation training and find best parameter

```
            y_prob = clf.predict_proba(X_val)[:, 1]
            score = roc_auc_score(y_val, y_prob)
            fold_scores.append(score)
        mean_score = np.mean(fold_scores)
        alpha_scores.append((alpha, mean_score))
        print(f"Alpha={alpha:.3f}, Mean AUC={mean_score:.4f}")
```

```
Alpha=0.010, Mean AUC=0.8285
Alpha=0.022, Mean AUC=0.8276
Alpha=0.046, Mean AUC=0.8267
Alpha=0.100, Mean AUC=0.8258
Alpha=0.215, Mean AUC=0.8253
Alpha=0.464, Mean AUC=0.8236
Alpha=1.000, Mean AUC=0.8218
Alpha=2.154, Mean AUC=0.8190
Alpha=4.642, Mean AUC=0.8137
Alpha=10.000, Mean AUC=0.8077
```

Cross-validation training and find best parameter

Find best parameter and train final model with best parameter

Cross-validation training and find best parameter

Find best parameter and train final model with best parameter

```
best_alpha, best_score = max(alpha_scores, key=lambda x: x[1])
print(f"Best alpha: {best_alpha:.3f}, AUC: {best_score:.4f}")
```

```
Best alpha: 0.010, AUC: 0.8285
```

```
best_model = BernoulliNB(alpha=best_alpha)
best_model.fit(X, Y)
```

Cross-validation training and find best parameter

Find best parameter by GridSearchCV

Explore other parameters of GridSearchCV

```
1 from sklearn.model_selection import GridSearchCV
2 param_grid = {'alpha': np.logspace(-2, 1, 10), 'fit_prior': [True,
        False]}
3 grid_search = GridSearchCV(BernoulliNB(), param_grid,
        scoring='roc_auc', cv=cv)
4 grid_search.fit(X, Y)
5 best_alpha = grid_search.best_params_['alpha']
6 best_fit_prior = grid_search.best_params_['fit_prior']
7 best_score = grid_search.best_score_
8 print(f"\nBest alpha: {best_alpha:.3f}, Best fit_prior:
        {best_fit_prior}, AUC: {best_score:.4f}")
```

```
Best alpha: 0.010, AUC: 0.8285
```